

Stage 1 Crash Course: Core Building Blocks

Everything else in system design (storage, caching, distributed systems, API design) is built on four primitives. Get these cold and every later tradeoff becomes a derivation instead of a memorized fact.

1. Scalability

Definition: the ability of a system to handle increased load by adding resources.

Vertical scaling ("scale up")

Add more power to an existing machine — more CPU, RAM, faster disk.

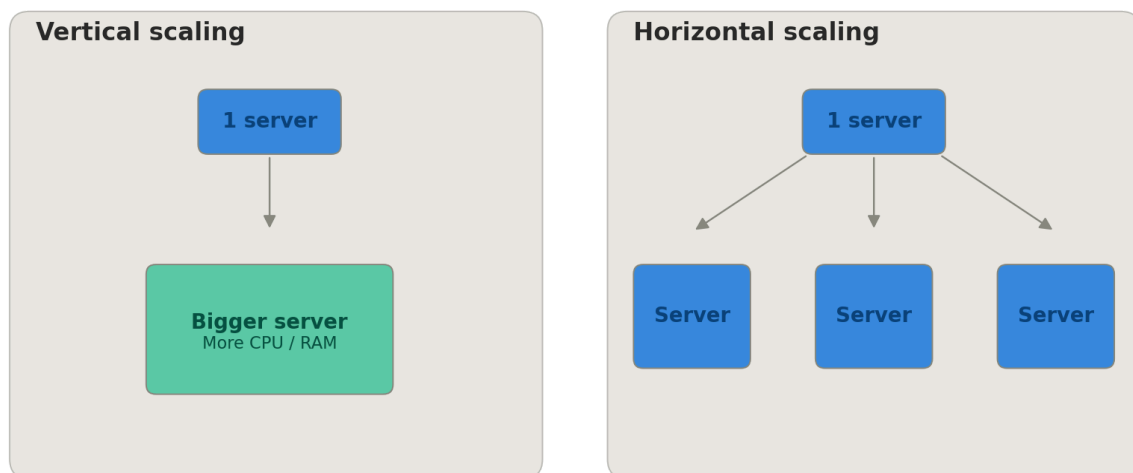
- Mechanism: more compute per request → same machine handles more load without architectural change.
- Ceiling: physical hardware limits, and cost grows super-linearly — a machine with 2x the cores/RAM typically costs more than 2x as much once you move past commodity hardware.
- Single point of failure: it's still one machine. If it dies, everything on it dies.
- Simplicity: no distributed systems complexity — one filesystem, one memory space, no network calls between "parts" of your app.

Horizontal scaling ("scale out")

Add more machines and split the load across them.

- Mechanism: a load balancer distributes requests across N identical machines → capacity grows roughly linearly with N (until the data layer becomes the bottleneck — see Stage 3).
- Ceiling: in principle, near-unlimited — but requires statelessness (or externalized state). If one machine holds something in memory another doesn't have, horizontal scaling breaks correctness, not just performance.
- Coordination overhead: load balancing, health checks, service discovery, and the entire distributed systems problem space (Stage 5).
- Redundancy for free: if one machine dies, the other N-1 keep serving.

Decision rule: default to horizontal for anything that needs to survive real production traffic and real hardware failures. Reach for vertical scaling first only when it's simpler and the ceiling is genuinely far away.

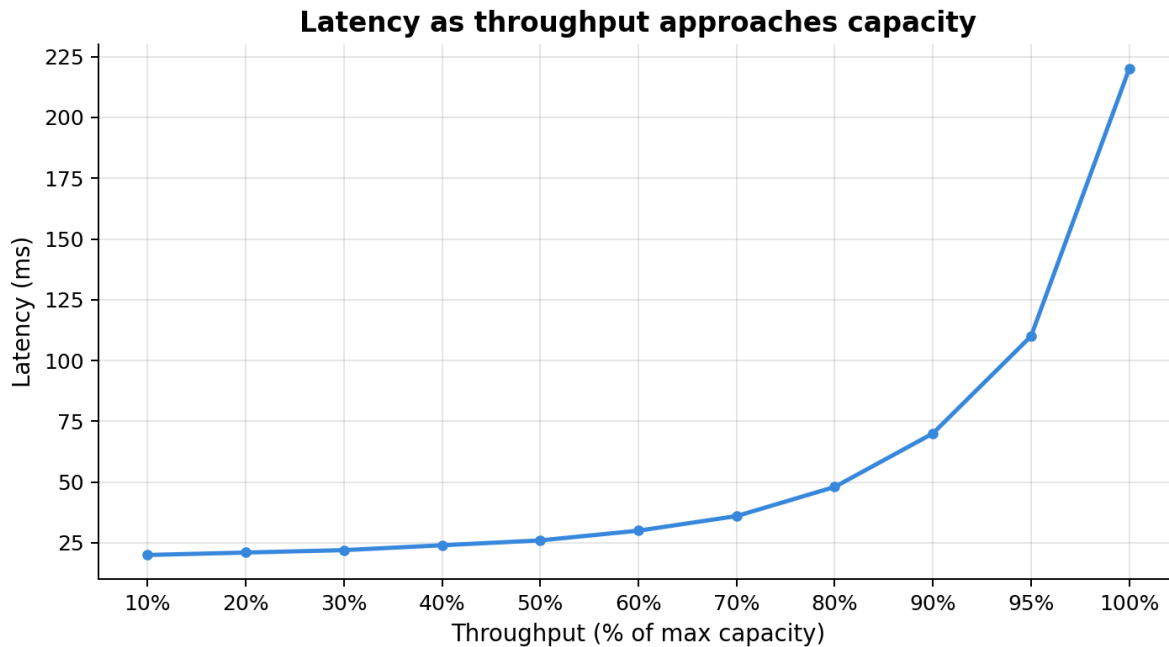


2. Latency vs. Throughput

Latency: time to complete one request (e.g., 40ms). **Throughput:** number of requests completed per unit time (e.g., 5,000 req/sec).

These are not simply inversely related — you can often improve both by adding parallel capacity. But they are coupled through queueing: as a system approaches its maximum throughput, requests start waiting in line, and latency rises — often sharply, not gradually.

Little's Law: $L = \lambda \times W$, where L = requests in flight, λ = arrival rate, W = average time per request. If arrival rate increases and processing rate doesn't, the number of requests queued grows, and so does the wait time each one experiences.



From 10% to 70% load, latency barely moves. From 80% to 100%, it more than quadruples — this is the queueing effect Little's Law predicts, and it's why a system can look completely fine in a load test at 70% capacity and then fall over the moment real traffic pushes it to 90%.

Why this matters: optimizing for throughput alone (e.g., batching requests) can directly increase latency for individual requests. This exact tradeoff shows up in ML model serving — batching improves GPU throughput but adds per-request latency, which is why serving systems expose a "max batch wait time" knob.

3. Reference Latency Numbers

You should have the order of magnitude of these memorized — not exact figures, but which is 1000x bigger than which.

Operation	Approx. latency
L1 cache reference	~1 ns
L2 cache reference	~4 ns
Main memory (RAM) reference	~100 ns
SSD random read	~100–150 μ s
Round trip within same data center	~0.5 ms
HDD disk seek	~10 ms
Round trip, cross-country	~40–60 ms
Round trip, cross-region / cross-continent	~100–150 ms+

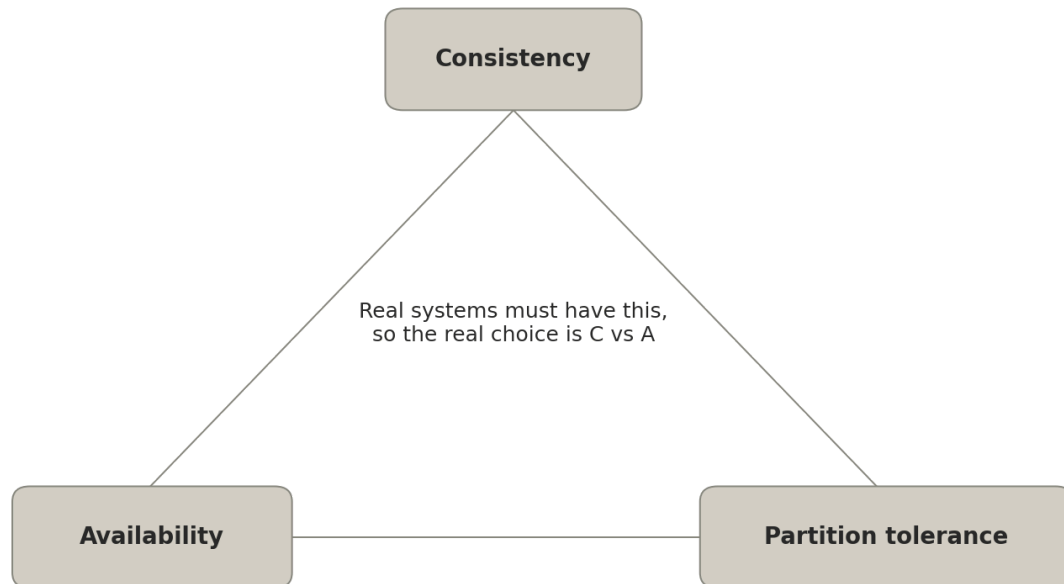
The intuition to walk away with: memory is ~100,000x faster than a cross-region network call. Every time you "just add a network hop" you're paying a cost that dwarfs almost anything happening inside a single machine — this is why caching, data locality, and minimizing cross-service chatter matter so much.

4. CAP Theorem (and why PACELC is the fuller picture)

In a distributed system, when a network partition occurs (some nodes can't talk to others), you must choose between:

- Consistency (C): every node returns the most recent write, or an error.
- Availability (A): every request gets a (non-error) response, even if it might not reflect the most recent write.

Partition tolerance (P) isn't really a choice — network partitions happen in any real distributed system, so every real system has to tolerate them. The actual decision CAP describes is: when a partition happens, do you fail closed (reject requests to stay correct) or fail open (keep answering, possibly with stale data)?



- CP systems (choose consistency): reject/block requests during a partition rather than risk returning stale data. Example: a bank balance check.
- AP systems (choose availability): keep answering during a partition, accept possible staleness, reconcile later. Example: a social media "like count."

PACELC extends this: If Partitioned, choose Availability or Consistency. Else (normal operation), choose Latency or Consistency. Even with a healthy network, strong consistency (every read confirms with a majority of replicas) pays a latency cost versus serving reads from the nearest replica without confirming. This is the tradeoff you're making on every single request, partition or not.

5. Reliability, Availability, and Fault Tolerance

- Reliability: the system performs its intended function correctly, consistently, over time.
- Availability: the percentage of time the system is operational and responding at all — says nothing about whether the response is correct.
- Fault tolerance: the system continues operating correctly despite the failure of some component, via redundancy and graceful degradation.

Availability in "nines"

Availability	Downtime / year	Downtime / month
--------------	-----------------	------------------

99% ("two nines")	~3.65 days	~7.3 hours
99.9% ("three nines")	~8.76 hours	~43.8 minutes
99.99% ("four nines")	~52.6 minutes	~4.4 minutes
99.999% ("five nines")	~5.26 minutes	~26 seconds

Why this matters: each additional nine is roughly an order of magnitude more expensive to achieve. Part of requirements clarification (Stage 9) is figuring out how many nines the problem actually needs — over-engineering wastes time and money, under-engineering is a production incident.

How fault tolerance is actually achieved

- Redundancy: multiple instances of every critical component.
- Health checks + failover: a load balancer or orchestrator detects a dead/unhealthy instance and stops routing to it.
- Graceful degradation: serve a degraded response (stale cache, default recommendation) instead of a hard failure.

How the four pieces connect

- Scalability decides how many copies of a component you have.
- Latency/throughput decide where those copies should live and how requests should be routed.
- CAP/PACELC decide what guarantee you give up when copies disagree or can't reach each other.
- Reliability/availability is the outcome — produced by getting the first three right.

Must be able to explain before moving on

1. Why horizontal scaling requires statelessness, with a concrete example of state that would break it.
2. Why "add a network hop" is never a free architectural decision — tie it back to the latency table.
3. Given a network partition, argue for a CP choice for one type of system and an AP choice for another, and defend both.
4. The difference between "available" and "reliable," with an example of a system that is one but not the other.